@ codedsoul_05

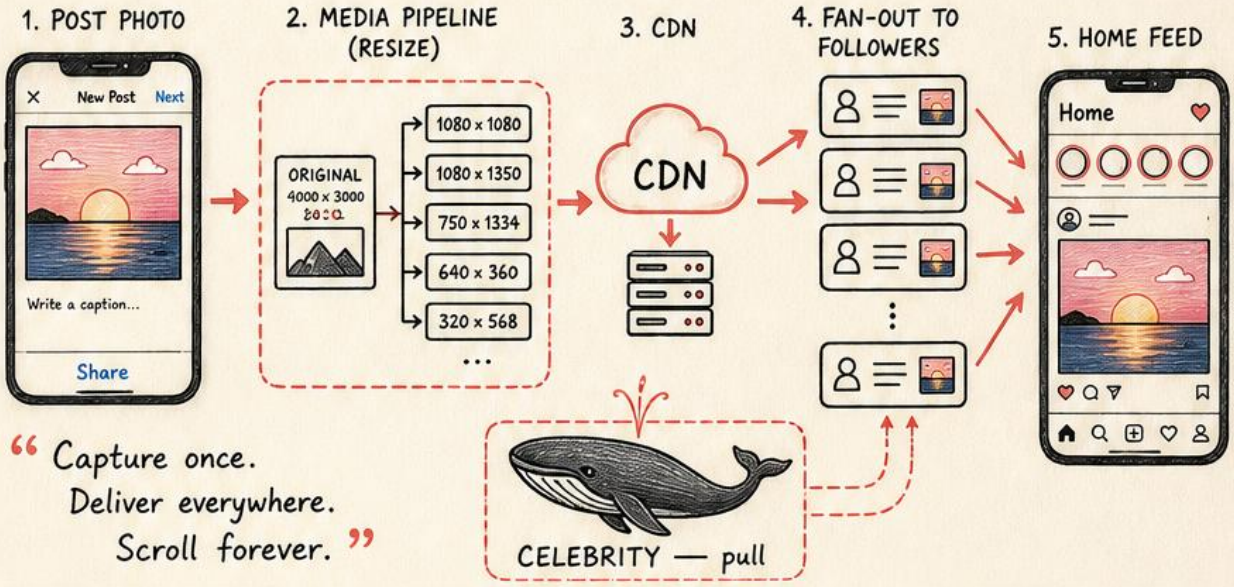
SOCIAL MEDIA SD v1.0

INSTAGRAM SYSTEM DESIGN

— HANDWRITTEN HANDBOOK —

Designing Feed, Stories, Media Pipeline & Celebrity Fan-out





"Capture once.
Deliver everywhere.
Scroll forever."

UPLOAD • FOLLOW GRAPH • FEED • STORIES • EXPLORE • CDN

WHAT'S INSIDE

1. Problem & Goals
2. High Level Design
3. Core Components
4. Data Model
5. Media Pipeline Deep Dive
6. Feed Ranking & Fan-out
7. Scalability & Performance
8. Reliability & Consistency
9. Monitoring & Metrics
10. Trade-offs & FAQs

INSTAGRAM BY THE NUMBERS

2+ BILLION MONTHLY USERS

PHOTO-HEAVY
90%+ POSTS ARE IMAGES OR VIDEOS

READ >> WRITE
HIGH CONSUMPTION, LOW CREATION

SKEWED FOLLOWS
TOP 1% ACCOUNTS DRIVE MAJORITY OF REACH

TOPICS COVERED

- ☑ Media Upload & Processing
- ☑ CDN & Edge Delivery
- ☑ Feed Generation & Ranking
- ☑ Fan-out Architectures (Push, Pull, Hybrid)
- ☑ Caching Strategies
- ☑ Scaling Follow Graph
- ☑ Stories & Ephemeral Content
- ☑ Explore & Recommendations
- ☑ Observability & SLIs/SLOs
- ☑ Cost Optimization

WHO IS THIS FOR?

- ☆ Backend & Platform Engineers
- ☆ System Design Learners
- ☆ Tech Leads & Architects
- ☆ Anyone building social products

"Twitter is text fan-out. Instagram is media fan-out with a heavier byte tax."

☆ UPLOAD FAST. FAN-OUT SMART. CACHE HOT.

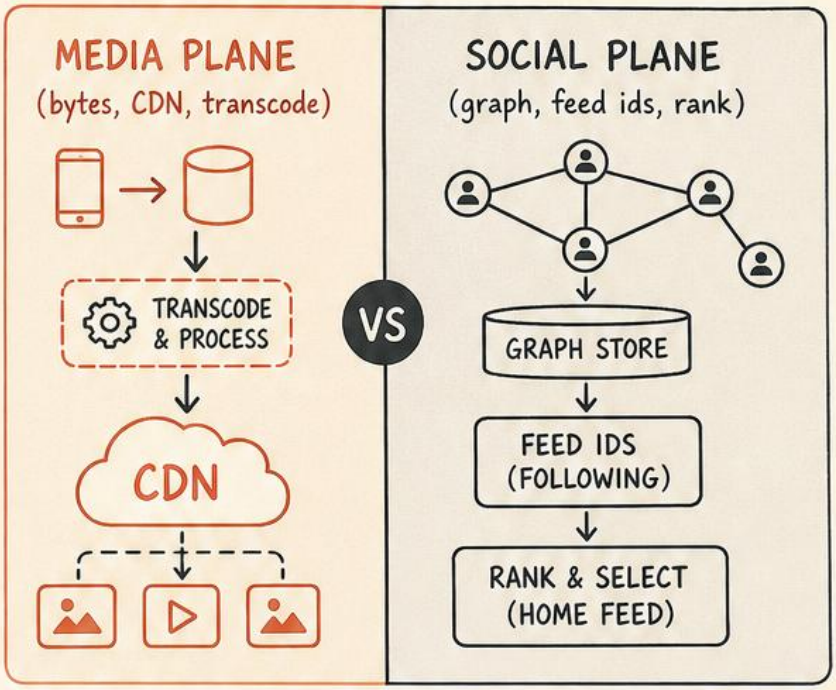
01 — REQUIREMENTS & SCOPE

GOAL | Let anyone share photos/videos and see a fast, personalised feed of people they follow.

CORE FEATURES

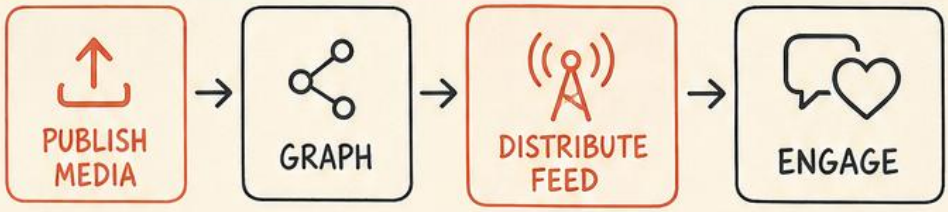
- Upload photo/video
- Follow / Unfollow
- Home feed
- User profile grid
- Like / Comment
- Stories (24h)
- Explore / Reels overview
- Notifications
- Search users / hashtags

TWO PLANES



★ NOTE: Separate them on the whiteboard immediately.

FUNCTIONAL BUILDING BLOCKS



NON-FUNCTIONAL REQUIREMENTS

- Upload reliability (no duplicate loss)
- Feed latency p95 < 500ms
- CDN hit rate > 90%
- Eventual consistency OK
- High availability (multi-region)
- Abuse resistance (spam, bots, content)

OUT OF SCOPE

- Full ads auction system
- Shopping / checkout deep dive
- VR / Metaverse
- WhatsApp

TIP

Say celebrity hybrid early — same skew as Twitter, but every post is a multi-MB object.

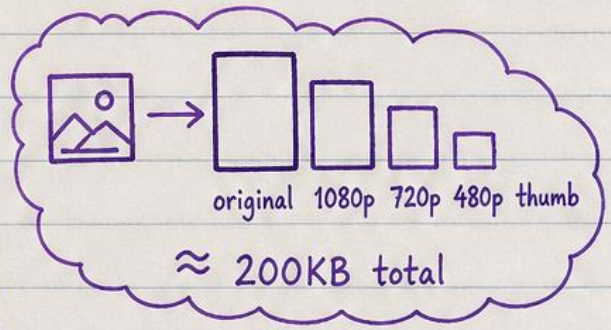
★ IDs in feeds. Bytes on CDN. Rank optional.

02 — Capacity Estimation

Reads dwarf writes. Media dominates storage and egress.

Assumptions:

- DAU = 500M
- Posts/day = 100M
- Avg followers = 200 (skewed)
- Feed refreshes/DAU/day = 20
- Avg photo processed sizes total ~200KB equivalents across variants
- Video heavier separately



① WRITE:

100M / 86400 ≈ 1,160 posts/sec avg;
peak several thousand

② FAN-OUT NAIVE:

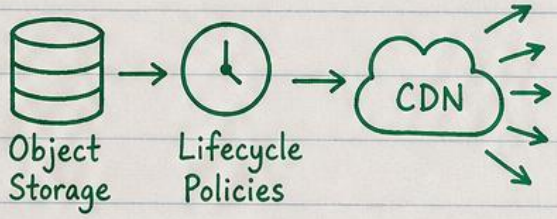
100M × 200 = 20B feed inserts/day
≈ 230k/sec — impossible for celebs → hybrid

③ READ:

500M × 20 = 10B feed reads/day
≈ 115k/sec avg — cache mandatory

④ STORAGE:

- 100M × 2MB raw-ish → 200TB/day class before dedupe/variants
- Object storage + lifecycle
- CDN egress enormous

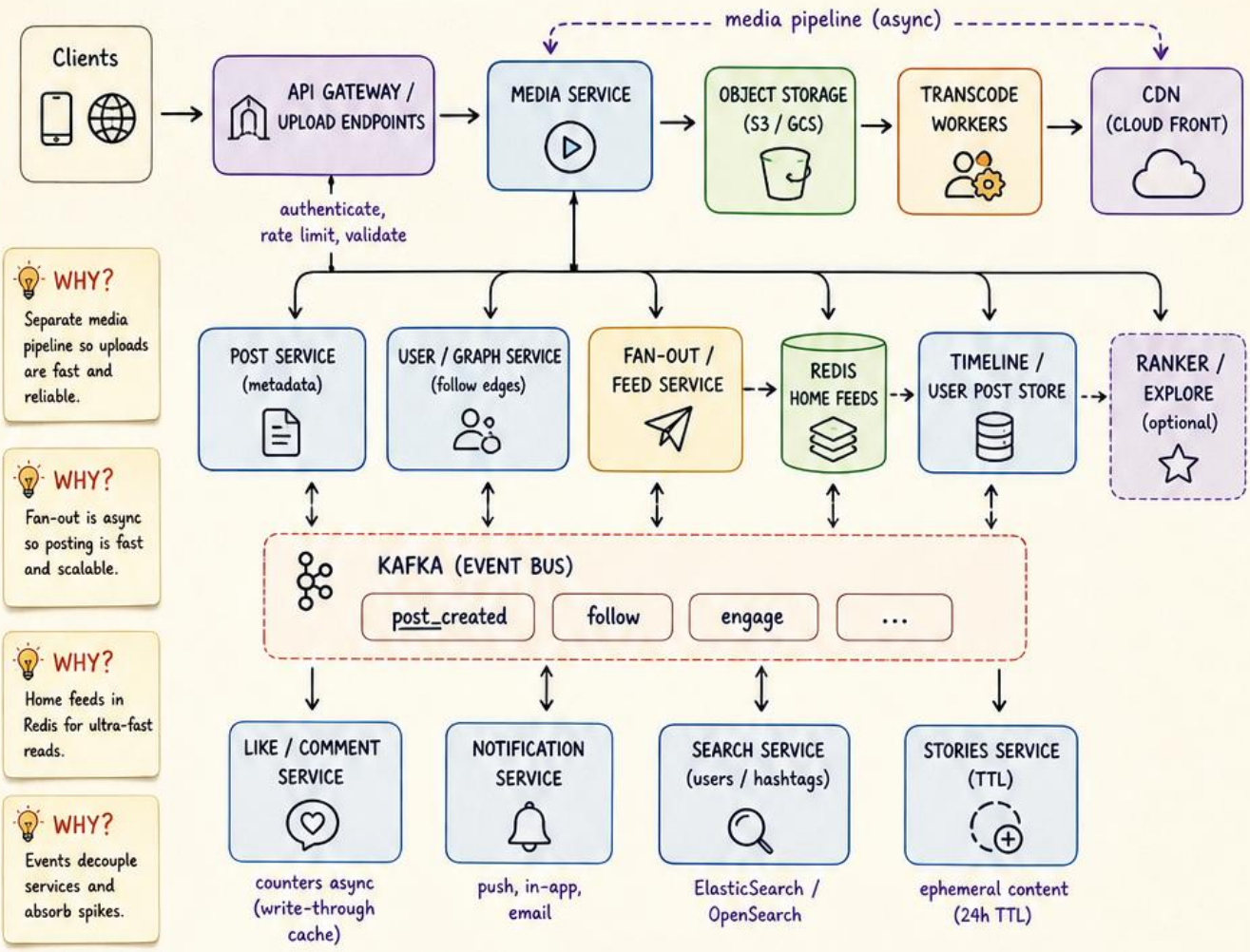


TAKEAWAY:

- Async upload processing
- Redis feed of post_ids (not the media)
- CDN for media
- Celebrity pull (fan-out on read for the famous)

★ Cache the feed IDs. Ship bytes from the edge.

03 — High-Level Architecture



WHY? Separate media pipeline so uploads are fast and reliable.

WHY? Fan-out is async so posting is fast and scalable.

WHY? Home feeds in Redis for ultra-fast reads.

WHY? Events decouple services and absorb spikes.

WHY? Counters & notifications are async to keep write path small.

POST PATH (WRITE)

ACK BEFORE FULL FAN-OUT



Create Post (Clients) → API Gateway → Persist Metadata → Return Post ID (Immediate) → Fan-out + Feeds (Async) → Counters, Notifications, Search (Async)

TIP: Return post id quickly; media can finish processing asynchronously with placeholder.

READ PATH (READ)

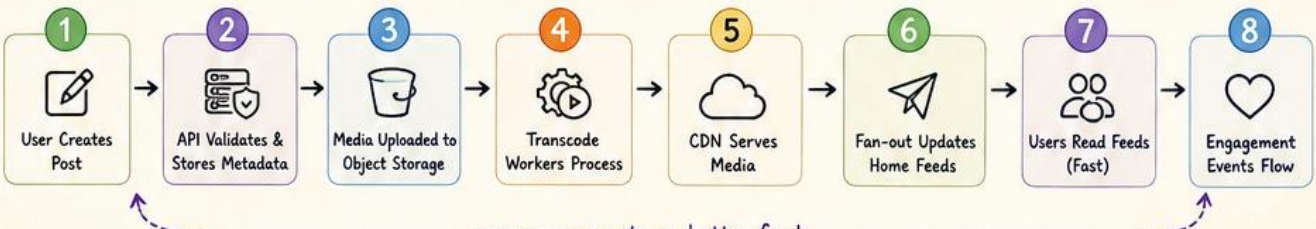
MERGE + HYDRATE + CDN URLS



Fetch Feed IDs (Redis) → Fetch Details (Post Store) → Merge + Enrich → Hydrate Media URLs (from CDN) → Render to Client

Reads are optimized: IDs from cache, details from store, media served via CDN.

END-TO-END FLOW (AT A GLANCE)



Metadata sync-ish. Bytes async. Feed async.

04 — APIs & CONTRACTS

Upload media first (or get signed URL).
Create post with media_ids.

MEDIA

- POST /v1/media/upload/init
- PUT chunks
- POST complete

- POST /v1/posts {media_ids, caption, location}
- DELETE /v1/posts/{id}

GRAPH

- POST/DELETE /v1/users/{id}/follow
- GET /v1/users/{id}/followers?cursor=

FEED

- GET /v1/feed/home?cursor=
- GET /v1/users/{id}/ posts?cursor=
- GET /v1/stories/tray ; GET /v1/stories/{user}

ENGAGE

- POST /v1/posts/{id}/likes
- POST comments

SEARCH

- GET /v1/search?q=

Create Post — JSON (example)

```
{
  "media_ids": [
    "01HM9Z3X5Q...A1B2C3",
    "01HM9Z3X5R...D4E5F6"
  ],
  "caption": {
    "text": "Sunset vibes 🌅"
  },
  "location": {
    "id": "01J2K3L4M5N6P7Q8R9"
  },
  "visibility": "public",
  "commenting_enabled": true,
  "sharing_enabled": true,
  "idempotency_key": "post_20240518_abc123"
}
```

POST /v1/posts

SIGNED UPLOAD URLs

Get short-lived signed URLs to upload directly to object storage (S3/GCS). Your servers never touch the bytes.

CURSOR PAGINATION

Use cursor= for stable pagination. The response includes next_cursor. Don't use offset.

next_cursor: "eyJ..."

RATE LIMITS

Per token and per IP. Respect headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset.

429 = Slow down

IDEMPOTENCY UPLOAD KEYS

Use idempotency_key when creating posts and media sessions to avoid duplicates on retries.

Safe to retry

Never upload multi-MB through your app servers if you can PUT to object storage directly.

Signed upload. Post metadata. Cursor feeds.

05 — DATA MODEL & STORAGE

Feed stores post IDs. Media lives in object storage. Graph is its own store.

ENTITIES

USER
- user_id (PK)
- handle
- name
- bio
- avatar_url
- privacy

POST
- post_id (PK)
- author_id (FK → USER.user_id)
- caption
- media_ids[]
- created_at
- location

MEDIA
- media_id (PK)
- type (image | video)
- variants { url, w, h }
- status (processing | ready | failed)

FOLLOW
- follower_id (FK → USER.user_id)
- followee_id (FK → USER.user_id)
- created_at (timeuuid)

HOME_FEED
- user_id (PK)
- post_ids (list<timeuuid>) // sorted desc by score/time
- updated_at

USER_POSTS
- author_id (PK)
- post_ids (list<timeuuid>) // sorted desc by created_at
- updated_at

STORY
- story_id (PK)
- user_id (FK → USER.user_id)
- media_id (FK → MEDIA.media_id)
- expires_at

LIKE
- user_id (PK → USER.user_id)
- post_id (FK → POST.post_id) // PK (user_id, post_id)
- created_at (timeuuid)

COMMENT
- comment_id (PK)
- post_id (FK → POSER.post_id)
- user_id (FK → MEDIA.user_id)
- text
- created_at (timeuuid)

COUNTERS (approximate)
- post_id (PK)
- like_count
- comment_count
- share_count
- updated_at

STORAGE & INFRA

Cassandra / Wide-Column
Posts, user_posts, home_feed, comments, counters

Redis (Hot Cache)
Hot home feeds, user sessions, rate limiting, counters cache

Graph Store (Sharded)
Social graph: follow relationships, mutuals, recommendations

S3 + CDN
Media objects & variants. Served via CDN at the edge.

Kafka
Ingest events (posts, likes, follows, comments), fan-out, pipelines

Elasticsearch
Search for users, captions, hashtags, locations

IDs
Snowflake IDs (time-ordered) for scale.

UUIDv7 for public-facing resource IDs.

Hydrate posts on read; never embed image bytes in feed rows.

ACCESS PATTERNS

Get Home Feed
Read user's home_feed (Redis → Cassandra fallback), hydrate posts.

User Profile
Read user → user_posts → hydrate posts.

Search
Query Elasticsearch, then hydrate users and posts.

Like / Unlike Post
Write like (Cassandra), update counter (async via Kafka).

Comment on Post
Write comment, push to Kafka, update counters.

Follow / Unfollow
Write to graph store + Cassandra, invalidate caches.

View Stories
Read stories by user, check expires_at, hydrate media.

IDs in timelines. Objects at the edge.

06 — MEDIA UPLOAD PIPELINE

Client uploads to blob storage; workers create size variants; CDN serves the bytes.

1. Client requests upload session / signed URL



Upload Service
Generates signed URL (time-limited, scoped)

2. Direct PUT to object storage (chunked for video)



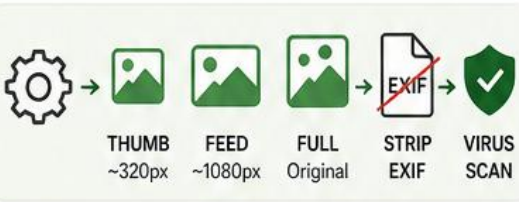
Blob/Object Storage
S3 / GCS / Azure Blob
Durable. Highly available.

3. Upload complete webhook → processing queue



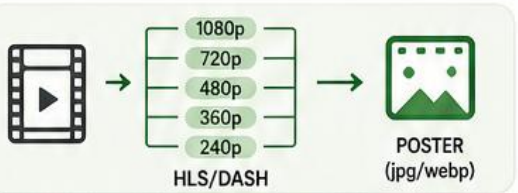
Event / Queue
SQS / Pub/Sub / Kafka
Decouples upload from processing.

4. Generate variants: thumb, feed width, full; strip EXIF; virus scan



Media Workers
Image pipeline (Sharp/Imagemagick)
ClamAV / Antivirus

5. Video: transcode ladder, posters, ABR



Video Workers
Transcode (FFmpeg)
ABR: HLS / DASH
Generate posters

6. Mark media READY → post can show; notify client



Metadata DB
Status = READY
Notify client via WS / Push / Webhook

7. CDN cache warm (optional for celebrity)



CDN
Pre-warm popular variants for faster global delivery

WHY NOT THROUGH API SERVERS?

Streaming large files through API servers wastes CPU, bandwidth, and adds latency.
Use direct-to-storage signed URLs.

IDEMPOTENT UPLOAD IDS

Use a client-generated upload_id (UUID).
If retried, server returns the same signed URL and final result.

PROCESSING FAILURE RETRIES

Workers should retry with backoff.
Store attempts & last error. Send to DLQ after max retries.

PROGRESSIVE JPEG/WEBP

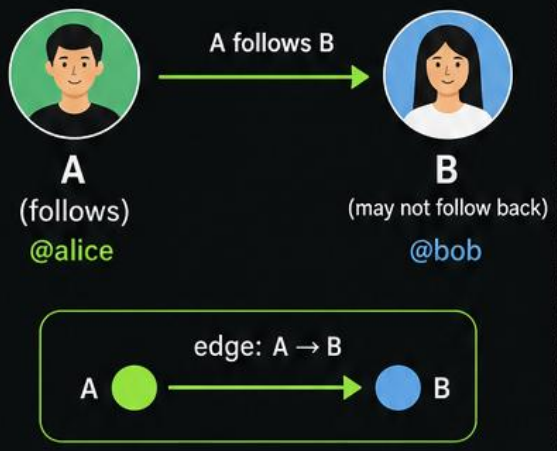
Serve progressive JPEG or WebP for images for faster perceived load.

Don't block create-post forever on 4K transcode — show processing state.
Let the user post immediately with a processing indicator.
Swap in final variants when READY.

Signed PUT. Async variants. CDN read path.

07 — FOLLOW GRAPH

Follow is one-way (unlike LinkedIn). Privacy modes change who can follow.



PUBLIC ACCOUNT

Anyone can follow. Edge is created immediately.



PRIVATE ACCOUNT

Follow request is sent. Target must approve.



- Store both: `following(user)` and `followers(user)` for fan-out vs profile
- Shard by `user_id`; chunk celebrity follower iteration
- Follow/unfollow events → `Kafka` for feed backfill / pull-set updates
- Private account: follow request pending; only `accept` adds edge
- `Block/mute` filters on feed

VS OTHERS

INSTAGRAM	TWITTER (X)	LINKEDIN
Follow is one-way	Follow is one-way	Connection is (mostly) two-way
Private accounts need approval	No private by default (anyone can follow)	Invitation + acceptance by both sides
Focus: content consumption	Focus: public conversation	Focus: professional networking

★ **Dual adjacency lists. Chunk the whale's followers.**

08 — FEED FAN-OUT ★ KEY PAGE

Push post_ids into followers' Redis feeds — except when follower count explodes.

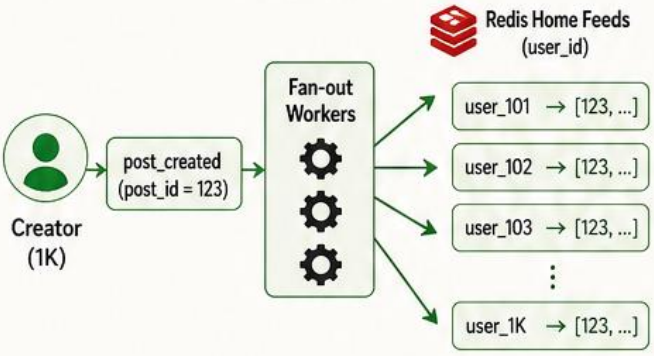
PUSH (fan-out on write):
on post_created, workers write post_id to each follower home list; great for read latency

PULL (fan-out on read):
keep posts on user timeline only; readers fetch on home request

HYBRID:
threshold T on follower count; below push, above pull

SMALL CREATOR (PUSH)

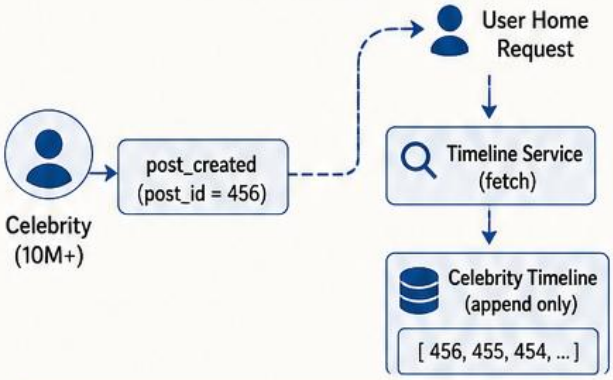
Followers: 1K



Write once, read fast from Redis home feeds

CELEBRITY (PULL)

Followers: 10M+



Read on demand from source timeline

ASYNC ACK: ACK the post API before fan-out finishes. Return 202 Accepted with post_id.

TRIM FEEDS: Keep only the last N post_ids per user (e.g., N = 1000) using Redis LPUSH + LTRIM.

BACKFILL ON FOLLOW: When user B follows A, backfill A's latest N posts into B's home feed. (or mark a cursor + lazy fill on read)

PITFALL:
Sync fan-out in request thread.

It will time out, retry storm, and take your system down.

★ **Push the many. Pull the mega.** ★

09 — HOME FEED READ PATH

Merge precomputed IDs + celebrity pulls → rank/filter → hydrate → attach CDN URLs.

1.  Read Redis home zset/list of post_ids
2.  Pull recent posts from celebrity followees' user timelines
3.  Merge by time (or ranking service)
4.  Filter muted/blocked/ deleted/seen
5.  Multiget post objects + authors
6.  Attach best media variant URLs for device
7.  Return page + cursor

CACHE MISS REBUILD

- Fan-out to followees (celebrities + close friends).
- Pull latest post_ids (from Redis zsets or timeline stores).
- Score/rank, filter, and write the merged post_ids into Redis home feed zset.
- Set page cursor (time + tie-breaker).

RANKING (MODERN IG RANKED FEED)

Affinity
People you interact with most.

Recency
Recent posts get higher weight.

Relationship
Close friends, DMs, tags, comments.

Negative Feedback
Hide, not interested, unfollow, report → downrank.



TIP

Say merge then hydrate — same as Twitter, plus media URL selection.



MERGE IDS. HYDRATE POSTS. SIGN CDN URLS.

10 – CELEBRITY / WHALE PROBLEM

100M followers × 1 post ≠ 100M Redis writes.

THRESHOLD MODEL

If follower_count > T, don't fan-out on write.
Switch to pull-based distribution.

PULL SET OF CELEBRITIES PER USER

Each user maintains a "pull set" of celebrity accounts they follow.

ON FOLLOW CELEBRITY → ADD TO PULL SET

When user follows a celebrity, add that celebrity to the user's pull set.

HOME MERGE FETCHES LAST K POSTS

On home read, fetch latest posts from the pull set (celebrities + regular follows) and merge, return top N.

SHARDED FAN-OUT FOR MID-TIER INFLUENCERS

For mid-tier accounts, still fan-out – but shard followers into chunks and rate limit the writes.

MEDIA STILL ONE OBJECT ON CDN – FAN-OUT IS ID DISTRIBUTION NOT BYTE DUPLICATION

We store the media once on CDN.
Fan-out only distributes post ID references.

LATENCY: CELEBRITY POST VISIBLE ON NEXT HOME READ WITHOUT WAITING FOR BILLION WRITES

No one waits for massive fan-out to complete.
It shows up on the next read.

COST MATH (BACK-OF-THE-NAPKIN)

NAÏVE FAN-OUT

100M followers × 1 post
= 100,000,000 writes
If 1 write = $0.000001
Cost ≈ $100 per post ☹

VS.

PULL MODEL

User reads home: fetch last K per celebrity
Assume 20 celebs × 20 posts = 400 IDs
Reads only. No massive writes.
Cost ≈ $0.0004 per read ☺

>250,000× CHEAPER!

IDs are fanned. Bytes are shared via CDN.

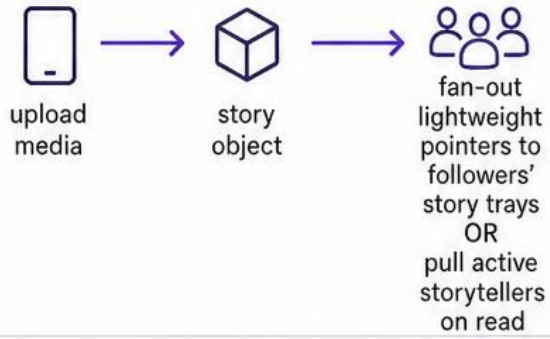
11 — STORIES (24-HOUR)

Stories are ephemeral feeds — TTL is a first-class feature, not a cron afterthought.

MODEL

- story items with expires_at = now+24h
- tray = list of users with unexpired unseen stories

WRITE PATH (UPLOAD)



READ PATH (VIEWER)



STORAGE & LIFECYCLE

- hot path **Redis** with TTL
- durable until expiry for safety
- auto-delete lifecycle on objects



RING UI STATE



Unseen
(has new story)



Seen
(no new story)

DIFFERENCES VS FEED

Higher write burstiness
Evenings spike writes.

Shorter retention
24h TTL vs days/weeks for feed.

Sequential playback
Linear, tap-to-next experience.

Contextual intensity
Designed for quick consume & reply.

TTL EVERYWHERE — TRAY, OBJECT, CDN CACHE SHORT.

12 – EXPLORE, REELS & RANKING

Following feed is retrieval from graph. Explore is candidate generation **outside your graph.**

1. CANDIDATES

- Embeddings (content & user)
- Topic Clusters (interests)
- Trending (global & local)
- Similar Accounts (follow graph hop)

2. RANK

Rank by P(engage)

Predict probability of meaningful engagement (dwell, interact, convert)

3. FILTER

Filter for

- Safety / policy
- Already seen
- Low quality / spam
- Age & regional restrictions

4. DIVERSIFY

Diversity & Balance

- Topics
- Creators
- Formats
- Freshness

RANKING SIGNALS

- Watch time / dwell
- Likes
- Saves
- Shares
- Skips / scroll past
- Author affinity (follows, history)
- Recency & freshness

REELS SPECIFIC



Video-first ranking

Optimized for session continuation and watch time depth.



Keep the session going.

OFFLINE TRAINING

Train models on historical interactions, dwell, and engagement outcomes.

NEARLINE FEATURES

Aggregate features, build embeddings, clusters, and signals.

ONLINE RANKER

Low-latency ranker scores candidates and returns the top N.

COLD START HANDLING

Show content that's popular in user's geo, language, or interest clusters.

Don't block upload path on ranker.

Ranking failures must fail-open.

Retrieve out-of-network, rank for dwell, filter junk.

13 — SEARCH: USERS, TAGS, PLACES

Typeahead for handles; inverted index for hashtags and captions.

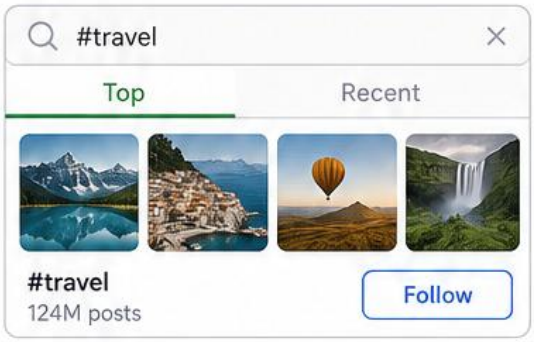
USER SEARCH

- prefix index on handle/name
- popularity boost (followers, engagement, recency)
- privacy filter (only show public accounts or permitted results)
- typeahead suggestions as you type



HASHTAG SEARCH

- tag → recent or top posts
- hot tags cached for speed
- spammy/abusive tag demotion
- ranking by recency, engagement, and relevance



PLACES / LOCATION SEARCH

- lighter index (city, place name, aliases)
- prefix match on place name
- return popular places and recent posts
- results scoped by user location (optional)



INDEXING PIPELINE

1. post_created (new post) → 2. extract hashtags from caption → 3. index into inverted index

All done async via message queue / workers

PERFORMANCE TARGET

Typeahead suggestions < 50–100ms from memory (trigram / prefix index)

- in-memory data structures (Redis / RAM)
- trigram index for fuzzy + prefix match
- LRU cache for hot queries and results

SCRAPING / RATE LIMITS

- respect platform rate limits
- backoff on 429 / temporary bans
- use rotating proxies ethically
- cache aggressively to reduce external calls

Prefix for people. Inverted index for tags.

14 — LIKES, COMMENTS & COUNTERS

Engagement is a write storm — shard counters and keep feeds ID-only.

LIKE

- idempotent (user,post) edge
- unlike deletes
- fan notifications aggregated

COMMENT

- append-only per post with cursor pages
- ranking sticky/top + recent

COUNTERS

- Redis/ counters service eventually consistent
- reconcile
- display 1.2M style

1.2M

QPS

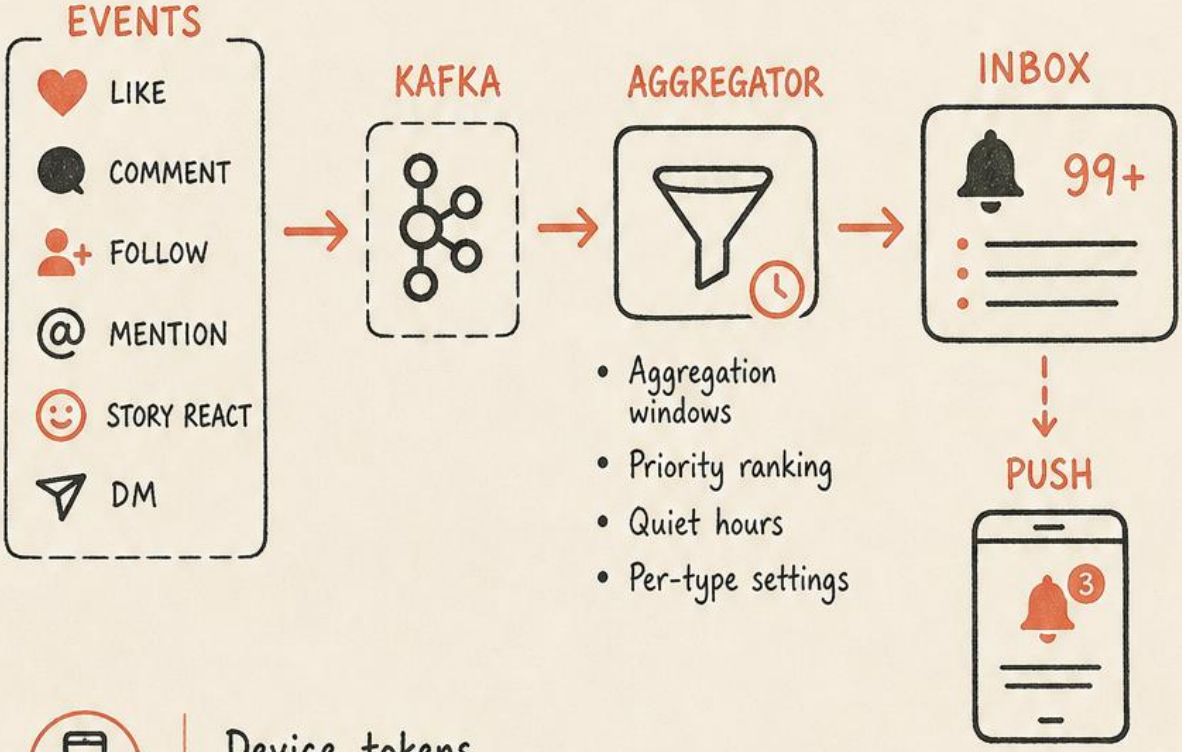
- viral post like spike — rate limit
- shard by post_id
- avoid single SQL row ++

Don’t put like fan-out
into every home feed rewrite.

**Idempotent edges. Approximate big counts.
Shard the viral post.**

15 — NOTIFICATIONS

Collapse bursts — "100 others liked your post" not 100 pushes.



Device tokens
Store per-device tokens (APNs, FCM).

Delivery at-least-once
Push may be retried; dedupe by notification ID.

Inbox cursor
Maintain per-user cursor for unread state.

Don't notify on your own actions
No notifications for actions you perform.

Mute / block honored
User mutes, blocks and preferences are respected.

Aggregate, prioritise, respect settings.

16 — SCALING, CACHE & CDN

The feed is an ID problem. The photo is a CDN problem.



SHARDS

- `user_id` for feeds/graph
- `post_id` for objects
- Redis cluster by user



CACHES

- home feed IDs
- post objects hot
- user profiles
- celebrity timelines replicated



CDN

- image/video variants at edge
- cache keys by `media_id+variant`
- high hit ratio on popular



MULTI-REGION

- media geo replicated
- feeds regional
- accept short lag



HOT KEYS

- viral posts, celebs — replicate reads, pull path for writes



Shard users. Replicate hot objects. Edge-cache bytes.

17 — FAILURES & CONSISTENCY

Own post should appear immediately.
Followers can lag seconds.

SCENARIO	WHAT WE DO
Fan-out lag	OK. Eventual delivery.
Redis miss	Rebuild from DB. Refill cache.
Processing stuck	Show “Retry” / “Try again”.
CDN miss	Fetch from origin. Cache & serve.
Delete (tombstone)	Scrub caches. Hide everywhere.
Kafka backlog	Scale consumers. Catch up.
Ranker down	Fallback to chronological.
Counter drift	Reconcile async. Correct over time.

CONSISTENCY MODEL

Read-your-writes for author;
eventual for others.

DEGRADATION PRINCIPLES

Skip explore. Skip rank.
Stories optional.

Durable post first. Fan-out catch-up second.

18 — ABUSE, SPAM & INTEGRITY

Bots, scrapers, and harmful content ride the same upload APIs.

CONTROLS

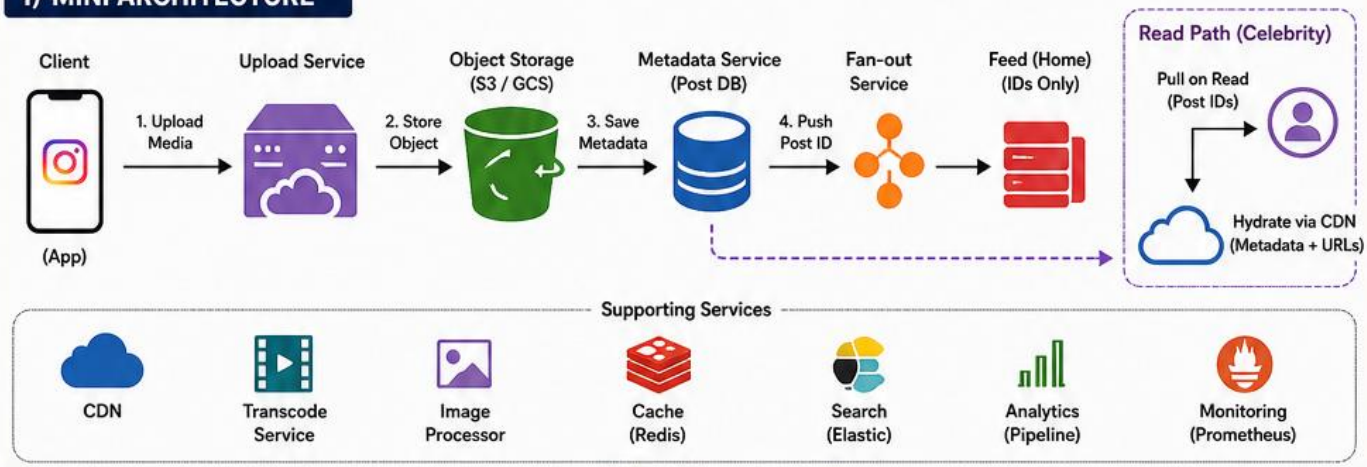
- rate limits upload/follow/like
- fake engagement detection
- hash matching CSAM pipelines (high level)
- spammy hashtags
- scraper throttling
- report → review
- shadow rank demotion
- Private accounts & block
- Stolen content / watermarking light

Throttle graph growth. Rank demote spam. Review reports.

19 — RECAP & INTERVIEW CHEAT SHEET

Upload media to **object storage**, store **post metadata**, **push IDs** to normal followers, **pull celebrities** on read, hydrate with **CDN URLs**.

1) MINI ARCHITECTURE



2) TOP 10 TALKING POINTS

1. Separate media (objects) from metadata (posts).
2. Object storage + CDN for durability, scale, and speed.
3. Fan-out post IDs to normal followers for fast feeds.
4. Pull model for celebrities to protect fan-out infra.
5. Feed DB stores only IDs, ranks, and small metadata.
6. Hydrate with CDN URLs on read, not on write.
7. Stories: short TTL, auto-expire, different storage tier.
8. Counters (likes/comments) via async increments.
9. Caching at multiple layers: client, CDN, Redis.
10. Design for scale, failure isolation, and cost efficiency.

3) VS TWITTER (X)

Aspect	Instagram	Twitter (X)
Media Weight	Heavy (images, videos, Reels)	Lighter (text, images, short videos)
Stories	Ephemeral (24h TTL)	No native stories (Spaces live)
Explore/Discover	Strong Explore + Reels algorithm	Timeline + Trends + Search
Feed Strategy	Ranked, diverse, interest-based	Ranked (latest + relevance)
Fan-out	Push IDs to normal followers	Fan-out tweets to followers
Celebrity Reads	Pull on read	Pull on read

4) INTERVIEW Q&A (QUICK HITS)

Q How do you handle celebrity accounts?
A Pull model on read to avoid fan-out explosion.

Q How do you ensure signed & secure uploads?
A Use pre-signed URLs with short TTL + scoped permissions.

Q How do stories expire after 24 hours?
A Store with TTL, background sweeper + lifecycle rules.

Q How do you handle like/comment counters?
A Async increments (Redis/Shard) + periodic persistence.

Q How do you serve media fast globally?
A CDN at edge + optimized formats + caching headers.

5) COMMON MISTAKES TO AVOID

Embedding bytes in feed DB	Bloats DB, kills performance, increases backup & restore cost.
Synchronous fan-out	Doesn't scale, causes latency, fails under spikes.
No hybrid strategy (push + pull)	Either overload fan-out or increase read latency.
Blocking on transcode	Slow upload UX; always queue and process async.
Ignoring TTL for stories	Storage bloat, stale content leaks, compliance issues.

IDS IN THE FEED. BYTES AT THE EDGE.